

Programação Funcional

Prática a02

Exploração inicial do ghci

```
2 + 3
5 * 10
2 ^ 8
True && False
not True
5 == 5
5 /= 3
succ 10
max 10 20
min 4 7
```

- `2+3`, `5*10` e `2^8` são expressões aritméticas
- `True && False`, `not True`, `5==5` e `5/=3` produzem valores booleanos: `True` ou `False`
- `succ`, `max` e `min` são funções já definidas no Haskell
- Em Haskell, os operadores também podem ser vistos como funções

Exploração inicial do ghci

```
:t 2 + 3
```

```
:t True
```

```
:t 'a'
```

```
:t "abc"
```

```
:t ['a', 'b', 'c']
```

- `:t` é um comando do GHCi para perguntar o tipo de uma expressão
- `'a'` é um Char
- `"abc"` é uma String
- uma String é uma lista de caracteres, isto é, `[Char]`

Primeiras funções

DOBRO

```
dobro x = 2 * x
```

A função recebe um valor `x` e devolve o dobro desse valor.

QUADRADO

```
quadrado x = x * x
```

A função recebe um valor `x` e multiplica esse valor por si próprio.

Primeiras funções

MÉDIA

`media x y = (x + y) / 2`

A função recebe dois valores e devolve a média aritmética.

POSITIVO

`positivo x = x > 0`

A função recebe um valor e devolve `True` se esse valor for maior do que zero; caso contrário, devolve `False`.

Testar

```
aulas > a02 > funcoes.hs
```

```
1 dobro x = 2 * x
2 quadrado x = x * x
3 media x y = (x + y) / 2
4 positivo x = x > 0
5
```

1. Criar um file com as funções
2. Load no ghci
3. Testar funções

```
(base) C:\Users\danie\Desktop\AULAS\pf2627\aulas\a02>ghci
GHCi, version 9.10.3: https://www.haskell.org/ghc/  :? for help
ghci> :load funcoes.hs
[1 of 2] Compiling Main                ( funcoes.hs, interpreted )
Ok, one module loaded.
ghci> dobro 10
20
ghci> quadrado 3
9
ghci> media 10 14
12.0
ghci> positivo (-5)
False
ghci> :t dobro
dobro :: Num a => a -> a
ghci> :t media
media :: Fractional a => a -> a -> a
ghci> :t positivo
positivo :: (Ord a, Num a) => a -> Bool
ghci> |
```

Exercícios

No mesmo ficheiro `funcoes.hs`, implementar e testar as seguintes funções.

- `triplo` Função que recebe um número e devolve o seu triplo.
- `metade` Função que recebe um número e devolve a sua metade.
- `areaRetangulo` Função que recebe a largura e altura de um retângulo e devolve a área.
- `areaCirculo` Função que recebe o raio de um círculo e devolve a área. Sugestão: usar `pi`.
- `maiorDeDois` Função que recebe dois valores e devolve o maior. Sugestão: usar `max`.

```
aulas > a02 >  funcoes.hs
```

```
1  triplo x = 3 * x
```

```
2  metade x = x / 2
```

```
3  areaRetangulo a l = a * l
```

```
4  areaCirculo r = pi * r^2
```

```
5  maiorDeDois a b = max a b
```

```
6
```

Ficha PL1 - Parte 1

no git <https://github.com/v2denny/pf2627>
pasta >aulas>a02>FichaPL1.pdf

Desafio

Implementa a função: `converteCelsius :: Double -> Double`

A função deve converter uma temperatura em **Fahrenheit** para **Celsius**.

Fórmula: $C = (F - 32) * 5 / 9$

Exemplos para testar:

- `converteCelsius 32` -- 0.0
- `converteCelsius 212` -- 100.0
- `converteCelsius 98.6`